Interactivity, State, and Hooks

At this point, React has taken over part of our page and is drawing the seats (although if you've looked at the before and after, you'll see that it was color-coding used tickets and it's not anymore, we'll bring that back in Chapter 11, *Managing State in React*). This is nice enough, but we'd like it to, you know, *react* to something. We'd like to have a little interactivity.

In React, you can use JSX to specify event handlers on React elements in much the same way you would when writing old-school JavaScript embedded in HTML. The problem is how to make changes to our components as a result of those events. As mentioned, the props we pass into each component are immutable, which means if we want to change something about a component, we can't use props. React uses the term *state* to refer to the parts of a component that change and trigger an update to how the component is displayed when they are changed.

To be clear, although a component can't change its own props, changing the state of a component can cause that component to re-render child components with new props.

Because state changes are used by React to trigger a redrawing of the page, React requires you to register them with the system; you can't just change the value of a variable and be done with it. React allows you to designate a value as being part of the state and gives you a special setter for that value using a mechanism called *hooks*.

Hooks are new in React as of version 16.8. Before then, components defined as functions could not manage changing state (components defined as classes always could manage state using a different mechanism). As mentioned earlier, the React core team has said that hooks and functional components are the way of the future, which is why we will be focusing on using hooks to manage state in this book.